

Context-Agent: Dynamic Discourse Trees for Non-Linear Dialogue

Junan Hu, Shudan Guo, Wenqi Liu, Jianhua Yin, Yinwei Wei*

Shandong University, China

junanhu@mail.sdu.edu.cn, weiyinwei@hotmail.com

Abstract

Large Language Models demonstrate outstanding performance in many language tasks but still face fundamental challenges in managing the non-linear flow of human conversation. The prevalent approach of treating dialogue history as a flat, linear sequence is misaligned with the intrinsically hierarchical and branching structure of natural discourse, leading to inefficient context utilization and a loss of coherence during extended interactions involving topic shifts or instruction refinements. To address this limitation, we introduce Context-Agent, a novel framework that models multi-turn dialogue history as a dynamic tree structure. This approach mirrors the inherent non-linearity of conversation, enabling the model to maintain and navigate multiple dialogue branches corresponding to different topics. Furthermore, to facilitate robust evaluation, we introduce the Non-linear Task Multi-turn Dialogue (NTM) benchmark, specifically designed to assess model performance in long-horizon, non-linear scenarios. Our experiments demonstrate that Context-Agent enhances task completion rates and improves token efficiency across various LLMs, underscoring the value of structured context management for complex, dynamic dialogues. The dataset and code is available at [GitHub](#).

1 Introduction

The advancement of dialogue systems based on LLMs is pivotal for the efficacy of next-generation applications, including AI Agents and collaborative robotics, where the ability to maintain context-aware communication is fundamental to task completion and user engagement (Durante et al., 2024; Yao et al., 2024; Sun et al., 2026). Following the advent of LLMs’ context window expansion techniques, the capabilities for multi-turn dialogue have been significantly enhanced (Li et al., 2025).

*Corresponding author.

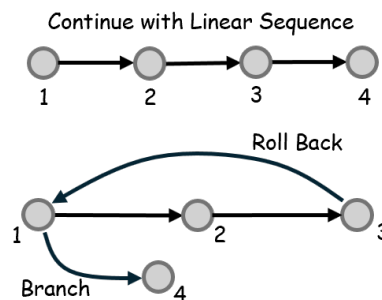


Figure 1: A schematic diagram of linear (upper) vs. non-linear (lower) dialogue flow.

However, LLMs still grapple with a fundamental challenge inherent to natural human conversation: the management of non-linear dialogue flow. This phenomenon occurs when conversational topics do not advance in a sequential order but instead feature shifts, topical jumps, or interwoven threads of discussion (Laban et al., 2025). Such non-linear dynamics are commonplace in real-world interactions, where users may revisit previous topics, introduce new subjects, or refine earlier statements based on evolving understanding or context (Mann and Thompson, 1988). The prevalent approach of treating dialogue history as a flat, linear sequence is fundamentally misaligned with the intrinsic structure of human conversation (Wang et al., 2024; Li et al., 2025). This linear paradigm fails to capture the hierarchical and branching nature of dialogues, leading to inefficiencies in context utilization and challenges in maintaining coherence over extended interactions (Lian et al., 2026; Ding et al., 2024).

Effectively resolving the non-linear flow problem requires overcoming several challenges. The first is the accurate identification and management of topic shifts and instruction refinements within a conversation. The second is the efficient selection of context from a potentially vast and complex dialogue history. As conversations extend over multiple turns, the accumulation of information can

lead to increased computational costs and the risk of overwhelming the model with irrelevant details (Joren et al., 2025; Jiang et al., 2026), leading to the “needle in a haystack” problem (Liu et al., 2024b; Vaswani et al., 2017). The third challenge lies in the development of robust evaluation metrics and benchmarks that can accurately assess a model’s performance in handling non-linear dialogues, as existing datasets often lack the complexity and variability found in real-world interactions.

To address these challenges, inspired by the hierarchical organization inherent in human cognitive processes for managing complex dialogues (Grosz and Sidner, 1986), we propose Context-Agent, a novel framework that models multi-turn dialogue history as a dynamic tree. This approach allows for the representation of conversations in a way that reflects their inherent non-linear nature, enabling the model to maintain multiple branches of dialogue corresponding to different topics. Furthermore, recognizing the inadequacy of existing datasets for this problem, we introduce the Non-linear Task Multi-turn Dialogue (NTM) benchmark, specifically designed to evaluate the performance of models in long-horizon, non-linear dialogue scenarios. This benchmark features dialogues with multiple topic shifts and instruction refinements, providing a more realistic and challenging testbed for assessing context management strategies.

In summary, the main contributions of this paper are as follows:

- We propose **Context-Agent**, a novel framework that models dialogue history as a dynamic tree. This approach captures non-linear discourse structure, enabling precise context navigation via tree structure.
- We introduce the **Non-linear Task Multi-turn Dialogue (NTM)** benchmark. It features long-horizon dialogues with complex topic shifts and instruction refinements, offering a rigorous testbed for non-linear context management.
- Experiments across various LLMs demonstrate that Context-Agent significantly outperforms linear baselines, improving task completion rates while reducing token usage.

2 Related Works

Linear Context Extension and Compression. While recent works have explored structured and task-aware parameter-efficient fine-tuning (Xiao

et al., 2026), architectures for context extension like YaRN (Peng et al., 2024) and LongLoRA (Chen et al., 2024) extend context windows but face high computational costs and the “lost-in-the-middle” problem (Liu et al., 2024b). Conversely, compression methods (Su and Zhou, 2022; Park et al., 2021) reduce token usage but degrade performance by flattening dialogue structure, sacrificing details essential for complex reasoning.

Structured Memory and Retrieval. Retrieval-Augmented Generation (RAG) adapts external retrieval to internal dialogue history, with various methods addressing data quality and mitigating retrieval-induced hallucinations (Zhang et al., 2026a; Ma et al., 2024). While flat retrieval methods like DH-RAG (Zhang et al., 2025) filter irrelevant turns, they often retrieve fragmented segments that lack local coherence. Recent advances have moved towards structured memory. Notably, MemTree (Rezazadeh et al., 2024) and RAPTOR (Sarathi et al., 2024) organize information into hierarchical tree structures.

Table 1 delineates the distinctions between our framework and existing paradigms. A fundamental limitation of current structured approaches, such as MemTree, lies in their reliance on **semantic similarity** for aggregation, grouping content based on textual overlap rather than **discourse flow**. This often conflates distinct conversational threads that share lexical features but diverge in intent. Conversely, **Context-Agent** explicitly models **discourse structure** (Grosz and Sidner, 1986). By constructing trees based on **navigational intent** (e.g., instruction refinement, topic switching) and retrieving coherent **paths** instead of isolated nodes, our approach preserves the logical continuity requisite for complex, long-horizon tasks.

3 Method

Our framework models a multi-turn dialogue as a forest of topic trees. Each tree represents a distinct topic and is composed of nodes (dialogue units) and branches. The dialogue’s evolution is managed through state transitions.

3.1 Formal Problem Definition

Conventional dialogue systems model history as a linear sequence $H_t = \{(q_1, r_1), \dots, (q_t, r_t)\}$, generating a response r_{t+1} from a query q_{t+1} via a function $g(H_t, q_{t+1})$. This flat representation leads to contextual redundancy and loss of structural in-

Method	Structure	Construction Basis	Retrieval Unit	Local Coherence	Update Efficiency
<i>Linear & Compression Methods</i>					
Full Context MemGPT	Linear Sequence OS-like Hierarchy	Token Concatenation Event-Triggered/Function	Entire History Paginated Memory	High High (Self-Edit)	Very Low ($O(N^2)$) Medium
<i>Retrieval-Augmented Generation (RAG)</i>					
Standard RAG DH-RAG	Flat Index Chain	Semantic Similarity Semantic Clustering	Indep. Chunks Query Chains	Low (Disjointed) High (Dynamic)	High Medium (Incremental)
<i>Tree-Structured Memory</i>					
RAPTOR MemTree	Static Tree Dynamic Tree	Bottom-up Clustering Online Clustering	Abstractive Summaries Collapsed Nodes	High Medium (Disjointed)	Low (Offline Rebuild) High ($O(\log N)$)
Context-Agent (Ours)	Dynamic Tree	Discourse Intent	Coherent Path	Very High (Path-Aware)	High (Event-Triggered)

Table 1: Comparison of context management paradigms. We compare our method with linear methods, standard RAG, advanced RAG, and tree-based memory.

formation.

To address this limitation, we introduce and formalize the problem of Non-linear Contextual Dialogue Management. The central premise of this problem is to shift from treating the entire history H_t as an undifferentiated input to representing it as a dynamically evolving, hierarchically structured dialogue forest, denoted as F_t .

We model the interaction flow as a dynamic tree to align with the Attentional State theory (Grosz and Sidner, 1986). This theory posits that human cognitive focus operates hierarchically, managing a focus stack rather than a connected graph. Explicit graph structures risk violating local coherence by merging distinct branches, thereby introducing noise from competing contexts. In contrast, our tree framework enforces logical isolation between diverging paths (e.g., separate travel plans). This design mirrors human cognitive separation, ensuring the model maintains a clear, distraction-free train of thought.

At each turn $t + 1$, given:

- A structured dialogue history represented as a forest, $H_t = F_t$.
- The current state $S_t = (H_t, T_{\text{act}}, B_{\text{act}}, n_{\text{cur}})$, which includes the history, the active topic tree, the active branch, and the current node.
- The new user query q_{t+1} .

The objective is to learn a policy π that comprises two key functions: a context selection function, f_{select} , and a response generation function, f_{gen} :

$$C_{t+1} = f_{\text{select}}(q_{t+1}, S_t)$$

$$r_{t+1} = f_{\text{gen}}(q_{t+1}, C_{t+1})$$

Here, C_{t+1} represents a highly relevant context subset, which is dynamically selected and constructed from the structured history H_t . The ultimate goal

is to maximize the task completion rate while minimizing the token footprint of the selected context C_{t+1} , thereby achieving efficient context utilization without compromising conversational coherence or task-oriented performance.

3.2 Core Components

Node The smallest unit of a conversation is a node n , which represents the content of a round of dialogue between the user and the model. Each node is defined as a tuple:

$$n = (c, v, p, \beta, s_i)$$

where c is the content of the current conversation round, $v \in \mathbb{R}^d$ is its d -dimensional text embedding, p is the parent node’s identifier (null for a root), β is the branch identifier, and s_i is a summary of the node’s content. After each round, a summarization function S_{node} converts the content c_i into a summary $s_i = S_{\text{node}}(c_i)$, which is used for subsequent topic attribution and branch management.

Topic Tree An independent topic is represented by a topic tree T . It is a directed acyclic graph, $T = (N, E)$. Here, $N = \{n_1, n_2, \dots, n_k\}$ is the set of all nodes under this topic, and $E = \{(n_i, n_j) \mid p(n_j) = n_i\}$ is the set of directed edges between nodes, representing the inheritance relationship of the conversation. The first dialogue round of a new topic is set as the root node, whose parent node is null, of the topic tree.

Branch Within the same topic tree T , a branch B is a relatively independent dialogue path that starts from a branching point but still remains under the same topic. It is defined as an ordered sequence of nodes $B = \langle n_1, n_2, \dots, n_k \rangle$, where any two adjacent nodes (n_i, n_{i+1}) in the sequence satisfy $p(n_{i+1}) = n_i$. All nodes within the same branch share the same branch identifier β .

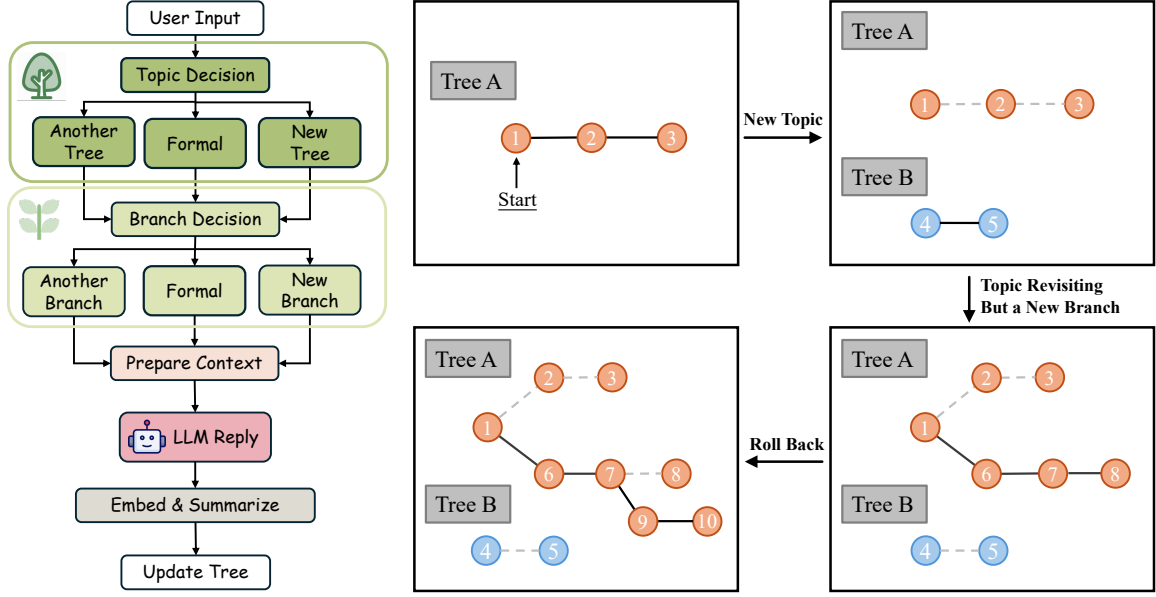


Figure 2: An overview of the Context-Agent framework. It illustrates the dynamic evolution of a multi-turn dialogue represented as a forest of topic trees, with branches indicating sub-dialogue paths. The number in each node represents the turn number in the conversation. Solid edges represent the active path, while dashed edges indicate inactive paths.

Conversation History The complete history H of a multi-turn conversation is represented as a forest F consisting of multiple topic trees, i.e., $H = F = \{T_1, T_2, \dots, T_m\}$.

3.3 State Transition

The conversational state at turn t is defined as $S_t = (H_t, T_{act}, B_{act}, n_{cur})$, which includes the history, the active topic tree, the active branch, and the current node. The conversation evolves through state transitions driven by new user queries. Upon receiving a new query, the system analyzes it to determine the topic and manage branches, updating the state accordingly. This process involves the following steps:

- **Step0: Initialization** Initialize the first topic tree T_1 as the active tree T_{act} . Define an aggregation function S to summarize branches or trees by concatenating their constituent node summaries (e.g., $S(B) = \text{Concat}(s_1, \dots, s_k)$).
- **Step1: Topic Decision** Given query q_{t+1} , a lightweight model Ψ determines the action a_{topic} and target tree T_{target} using existing tree summaries:

$$(a_{\text{topic}}, T_{\text{target}}) = \Psi(q_{t+1}, \{S(T_i)\})$$

T_{act} is updated to T_{target} . Actions include:

- **CREATE_TOPIC**: Start a new topic tree.

- **SWITCH_TOPIC**: Switch to an existing tree.
- **CONTINUE**: Stay in the current tree.

- **Step2: Fork Point Identification** For a new query q_{t+1} , the system first computes its embedding vector $v_{q,t+1} = \epsilon(q_{t+1})$ using the embedding function $\epsilon : C \rightarrow \mathbb{R}^d$. Then, among all nodes in the active topic tree T_{act} , it identifies the node most semantically relevant to q_{t+1} as the potential fork point. This is achieved by maximizing the similarity function $\text{Sim}(v_{q,t}, v_i)$:

$$n_{\text{fork}}^* = \arg \max_{n_i \in N_{act}} \text{Sim}(v_{q,t+1}, v_i)$$

- **Step3: Branch Decision** Branch decision employs a two-stage “heuristic filtering + model decision” approach. First, a heuristic function H_{filter} quickly determines if a complex decision is needed. Specifically, H_{filter} returns true if the most similar node n_{fork}^* found in Step 2 is sufficiently relevant and it either belongs to a different branch or is an ancestor of the current node.

If H_{filter} is true, a lightweight language model Φ determines the branch action a_{branch} based on the query, current path, and retrieved nodes $R(q)$. Otherwise, the action defaults to CONTINUE.

$$a_{\text{branch}} = \begin{cases} \Phi(q_{t+1}, \text{Path}(n_{cur}), R(q_{t+1})) & H_{\text{filter}} \\ \text{CONTINUE} & \neg H_{\text{filter}} \end{cases}$$

The possible actions are:

- **CONTINUE**: Add a new node to the branch.
- **CREATE_BRANCH**: Start a new branch from the fork point n_{fork}^* .
- **SWITCH_BRANCH**: Switch the active branch to the one containing n_{fork}^* .

- **Step4: Context Construction** The final context C_{t+1} is constructed by combining the full dialogue of the current active path with summaries of inactive branches and topics. This provides focused, relevant information while maintaining a broad overview of the entire conversation. The context is formed as:

$$C_{t+1} = \text{Concat}(\{c_i \mid n_i \in \text{Path}(n_{\text{cur}}, T_{\text{act}})\} \oplus \bigoplus_{\substack{B_j \in T_{\text{act}}, \\ B_j \neq B_{\text{act}}}} S(B_j) \oplus \bigoplus_{\substack{T_k \in H_t, \\ T_k \neq T_{\text{act}}}} S(T_k))$$

This structured context includes: (1) The complete dialogue history of the current active path. (2) Summaries of all other branches within the active topic tree. (3) Summaries of all other topic trees in the conversation history.

4 Non-linear Task Multiturn Dialogue (NTM) Benchmark

Existing multi-turn datasets typically feature short (<10 turns), linear contexts (Deshpande et al., 2025; Kwan et al., 2024; Bai et al., 2024), failing to capture the complexity of dynamic topic shifts essential for evaluating long-range reasoning. To bridge this gap, we introduce the Non-linear Task Multiturn Dialogue (NTM) benchmark.

4.1 Data Creation

NTM comprises a collection of dialogues focused on two domains: daily life planning and coding support. The dataset was constructed using state-of-the-art LLMs leveraging few-shot prompting to generate initial dialogues. Subsequently, each dialogue underwent a rigorous process of manual review, polishing, and filtering by human annotators to ensure high quality and task complexity.

Crucially, NTM dialogues focus on two significant aspects: Topic shifts and Instruction Refinement, which are common in real-world conversations but often overlooked in existing datasets.

- **Topic Shifts**: Each dialogue is designed to include multiple topic shifts. These shifts are contextually relevant, reflecting how real conversations evolve. For example, a dialogue may start

with planning a trip and then shift to discussing dietary preferences for the trip.

- **Instruction Refinement**: The dialogues also incorporate instances where users refine or change their instructions based on previous responses. This aspect tests the model’s ability to adapt to evolving user needs and maintain coherence throughout the conversation.

This design ensures that NTM evaluates not just information recall, but a model’s ability to maintain focus and adapt to a dynamically evolving conversational landscape.

4.2 Key Characteristics

NTM is distinguished by the following features:

- **Extended Dialogue Length**: The dataset includes a total of 405 dialogues with about 6900 turns, covering 10, 15, 20, and 25 rounds of conversations, which provide a clear measure of model scalability as context grows.
- **Topic Dynamics**: Each dialogue contains multiple topic shifts and instruction refinements, challenging models to maintain coherence and relevance in a non-linear conversational flow.
- **Task-Oriented Focus**: Every dialogue culminates in a clear task that requires accurate information synthesis from the preceding conversation, enabling objective evaluation through task completion metrics.

4.3 Evaluation Metrics

We evaluate the performance from 2 perspectives: task completion accuracy and token efficiency.

- **Task Completion Rate (TCR)**: Our primary metric for task success. Each task in the NTM benchmark is decomposed into at least three verifiable checkpoints(a yes/no decision). TCR is the average completion rate across these checkpoints, providing a robust measure of task fulfillment. This annotated metric provides a more robust and interpretable measure of a model’s true task-fulfillment capabilities compared to relying solely on scores from a judge LLM.
- **Average Context Tokens (ACT)**: Measures the average number of context tokens used per turn. It quantifies context efficiency, with lower values indicating better performance, which is crucial for managing long dialogues under token and cost constraints.

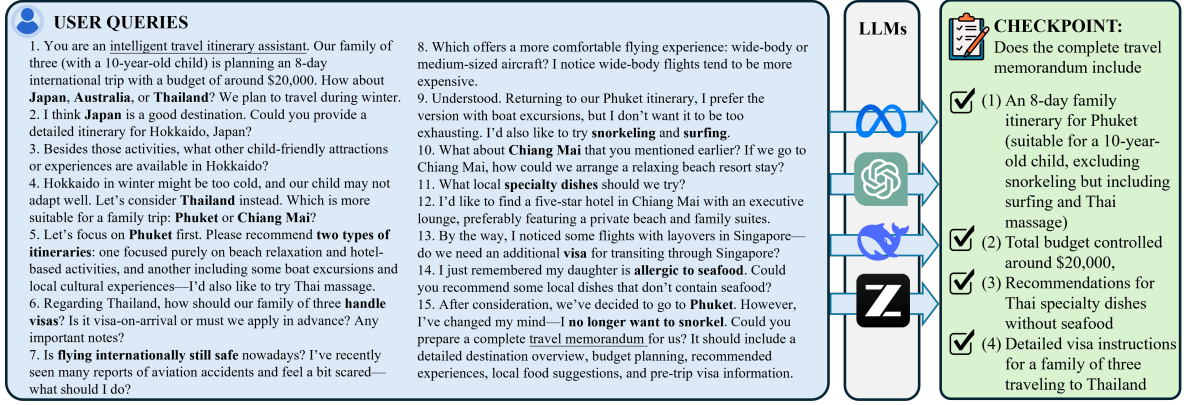


Figure 3: A 15-turn NTM dialogue example on trip planning, featuring topic shifts and instruction refinements. The right panel lists checkpoint questions for objective task completion evaluation. See Appendix A.6 for details.

4.4 Comparison with Existing Datasets

Table 2 compares NTM with existing datasets. NTM is distinguished by significantly longer turn counts and unique non-linear evolution, offering a more rigorous benchmark for complex dialogue evaluation.

Dataset	Avg. Turns	Max Turns	Total turns	Non-linear Evolution
Multichallenge	5	10	1365	No
MT-Eval	7	14	1170	No
MT-Bench-101	3	7	4208	No
NTM (Ours)	17	27	6931	Yes

Table 2: Comparison of NTM with existing multi-turn dialogue datasets.

5 Experimental Setup

We conduct a comprehensive evaluation to assess Context-Agent’s efficacy in managing long-form, non-linear dialogues, specifically examining its performance against baselines on complex tasks, its improvement in token efficiency relative to task success, and the distinct contributions of the tree-structured representation and retrieval mechanism.

5.1 Evaluation Benchmarks

A significant challenge in evaluating long-turn conversational models is the lack of suitable benchmarks. Existing datasets typically feature short, linear dialogues that do not adequately test a model’s ability to handle complex, evolving conversations. And the most important reason is that their context offered to the model is usually a fixed-length linear sequence, which cannot reflect the advantages of our Context-Agent in managing non-linear dia-

logue history. Therefore, all models are evaluated on our newly proposed Non-linear Task Multi-turn Eval (NTM) benchmark.

To evaluate the generalizability of our method on public datasets, we selected TopiOCQA (Adlakha et al., 2022) due to its rich topic shifts, which align well with our focus on non-linear dialogue management. We made appropriate adjustments to the dataset to facilitate testing within our framework, reporting Exact Match (EM) and F1 scores on the validation set.

5.2 Baseline Methods

We benchmark our Context-Agent framework against mainstream context management methods, which can be categorized into three groups:

- **Full History Concatenation (Full-History):** This method involves concatenating the entire dialogue history as input to the model. While it provides complete context, it is computationally expensive and often impractical for long conversations due to token limits.
- **Truncation (Truncation):** This approach retains only the most recent k turns of the conversation, discarding earlier context. It is efficient but risks losing important information from previous dialogue turns. In our experiments, we set $k = 4$.

Model	Open Source	Context Window
GPT-4.1	×	1000k
DeepSeek-V3	✓	64k
GLM-4-Plus	×	128k
Llama 3.1-70B	✓	128k

Table 3: Details of the LLMs used

To ensure a comprehensive evaluation of our Context-Agent across different models, we conducted experiments on four recent and diverse LLMs: GPT-4.1 (OpenAI, 2025), DeepSeek-V3 (Liu et al., 2024a), GLM-4-Plus (GLM et al., 2024), and Llama 3.1-70B (Grattafiori et al., 2024). This selection includes both open- and closed-source models with varying context window sizes. For fairness and efficiency, all evaluations were performed with reasoning-disabled settings.

5.3 Implementation Details

To balance processing efficiency and accuracy, we employ gemma3-12B (Team et al., 2025) for decision-making and gemma3-4B for summary generation. For dialogue context encoding, we use Qwen3-Embedding-0.6B (Yang et al., 2025). All experiments were conducted with an NVIDIA A100 40GB GPU. For evaluation, we adopt a triangulated protocol combining human annotators and Judge LLMs (GPT-5 and Gemini-2.5-Pro). For more details, please refer to Appendix A.2.

6 Results and Analysis

6.1 Main Results

The main results of our experiments are summarized in Table 4. Across all four LLMs, our Context-Agent consistently outperforms the Truncation method by a significant margin in terms of Task Completion Rate (TCR). Notably, our method not only recovers the performance loss caused by truncation but also surpasses the Full-History method across the board. Specifically, it achieves relative TCR improvements of 3.4%, 7.8%, 8.1%, and 9.7% on GPT-4.1, DeepSeek-V3, GLM-4-Plus, and Llama 3.1-70B, respectively. Even for GPT-4.1, which possesses a massive context window, Context-Agent achieves a score of 88.9%, outperforming the Full-History score of 86.0%. This suggests that structured context management effectively filters noise that can distract even the most capable models. Furthermore, Context-Agent demonstrates superior efficiency, reducing the Average Context Tokens (ACT) by approximately 45% to 52% compared to the Full-History approach. This dual advantage of higher accuracy and lower token consumption underscores the efficacy of the Context-Agent.

Table 5 demonstrates Context-Agent’s robust generalization on TopiOCQA. It outperforms Full-History in accuracy (EM/F1) while using only

~57% of the context tokens. This efficiency stems from the tree-structured memory, which isolates the active topic to minimize noise without losing necessary context.

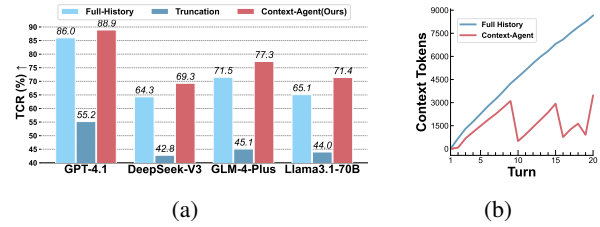


Figure 4: (a) TCR comparison across different methods and models. (b) A typical example of context tokens change trend in a 20-turn dialogue.

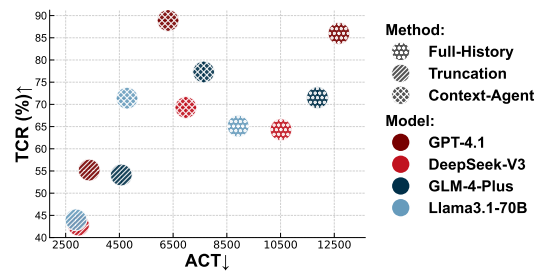


Figure 5: Trade-off between TCR and ACT, where the ideal point is the top-left corner (high TCR, low ACT).

Another notable observation is that though another 3 open-source models (DeepSeek-V3, GLM-4-Plus, and Llama 3.1-70B) still have considerable context windows (64k or 128k tokens), and the total context length of our NTM benchmark is lower than these limits, their TCR scores with Full-History are still significantly lower than that of GPT-4.1. This indicates that merely having a large context window does not guarantee effective utilization of context, especially in complex, non-linear dialogues. Our Context-Agent has demonstrated its ability to effectively manage and utilize context, leading to substantial performance gains.

From these results, we have several key insights:

- **Effectiveness of Context-Agent:** The consistent TCR improvements across different models and dialogue lengths demonstrate that Context-Agent effectively manages context in complex, long-horizon dialogues. It not only recovers the performance lost due to truncation but also surpasses the full-history approach in most cases.
- **Token Efficiency:** The significant reductions in ACT indicate that Context-Agent is highly effi-

Model	Method	TCR (%) $\uparrow$	TCR Gain (%)	ACT $\downarrow$				ACT Drop (%)
				10-turn	15-turn	20-turn	25-turn	
GPT-4.1	Full-History	86.0	–	4070	6382	9535	12803	–
	Truncation	55.2	-35.8	1839	2378	2981	3142	–
	Context-Agent	88.9	+3.4	2108	2894	4137	6227	-52.3
DeepSeek-V3	Full-History	64.3	–	3540	5428	7805	10693	–
	Truncation	42.8	-33.4	1732	2088	2535	2883	–
	Context-Agent	69.3	+7.8	1914	2873	4110	6014	-46.0
GLM-4-Plus	Full-History	71.5	–	4130	6996	9403	11782	–
	Truncation	45.1	-36.9	2890	3479	3783	4674	–
	Context-Agent	77.3	+8.1	1954	3027	4695	7032	-49.9
Llama 3.1-70B	Full-History	65.1	–	3540	5183	7189	8994	–
	Truncation	44.0	-32.4	1689	1898	2435	2860	–
	Context-Agent	71.4	+9.7	2075	2738	3843	4780	-45.5

Table 4: **Main Results on Context Management Efficiency and Effectiveness.** Performance on our proposed **NTM Benchmark** (Task-Oriented) across varying dialogue lengths. **TCR**: Task Completion Rate; **ACT**: Average Context Tokens. Context-Agent consistently outperforms baselines.

Method	EM(Exact Match)	F1 Score	ACT
Full-History	13.3	25.2	4261
Truncation	7.1	12.8	1703
Context-Agent	16.2	28.9	2435

Table 5: Result of Llama 3.1-70B on TopiOCQA.

Method	TCR (%)	TCR Drop (%)
Full-History	64.3	-
w/o Tree	41.5	-35.5
w/o RAG	45.3	-29.5
Context-Agent	69.3	+7.8

Table 6: Ablation study results on DeepSeek-V3.

cient in utilizing context. By intelligently selecting relevant information through its tree structure and RAG mechanism, it minimizes unnecessary token usage while still providing sufficient context for accurate responses.

- **Robustness Across Models:** The performance gains observed across a diverse set of LLMs, including both open-source and closed-source models with varying context window sizes, highlight the robustness and generalizability of the Context-Agent framework.

6.2 Ablation Studies

To isolate component contributions, we conducted an ablation study (Table 6). We evaluated two variants: (1) **w/o Tree**, which applies RAG to a flattened linear history (retrieving $k \in \{3, 5\}$ turns), and (2) **w/o RAG**, which relies solely on heuristics for branch decisions without semantic retrieval.

Results indicate that both components are essential. Removing the tree structure (**w/o Tree**) leads to a 35.5% TCR drop, confirming that linear retrieval captures semantic similarity but fails to maintain the logical flow necessary for effective context selection. Similarly, removing the retriever

(**w/o RAG**) results in a 29.5% drop, showing that heuristics alone are insufficient for accurate fork point identification.

7 Conclusion

In this paper, we addressed the critical limitation of conventional linear context management in handling the non-linear flow of multi-turn dialogues. We introduced Context-Agent, a novel framework that represents dialogue history as a dynamic tree structure, augmented by a retrieval mechanism. This approach successfully models the hierarchical and branching nature of human conversations, enabling effective navigation of complex interactions involving topic shifts and refinements. Our extensive experiments on the newly proposed NTM benchmark demonstrate that Context-Agent consistently outperforms traditional context management methods across various LLMs, achieving significant improvements in task completion rates while drastically reducing token usage. Ablation studies confirm the critical contributions of both the tree structure and RAG components to the overall performance. Our work underscores the potential of structured context management and offers

a promising direction for developing more robust and efficient dialogue systems capable of handling long-horizon, dynamic conversations.

Limitations

Current implementation relies on lightweight models for topic and branch decisions, whose performance may vary with model choice and prompting strategies. While our experiments show consistent gains across multiple backbones, further optimizing or learning these decision modules end-to-end could potentially yield additional improvements.

References

- Vaibhav Adlakha, Shehzaad Dhuliawala, Kaheer Suleman, Harm de Vries, and Siva Reddy. 2022. Topicqa: Open-domain conversational question answering with topic switching. *Transactions of the Association for Computational Linguistics*, 10:468–483.
- Ge Bai, Jie Liu, Xingyuan Bu, Yancheng He, Jiaheng Liu, Zhanhui Zhou, Zhuoran Lin, Wenbo Su, Tiezheng Ge, Bo Zheng, and Wanli Ouyang. 2024. Mt-bench-101: A fine-grained benchmark for evaluating large language models in multi-turn dialogues. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, ACL 2024, pages 7421–7454. Association for Computational Linguistics.
- Yukang Chen, Shengju Qian, Haotian Tang, Xin Lai, Zhijian Liu, Song Han, and Jiaya Jia. 2024. Longlora: Efficient fine-tuning of long-context large language models. In *The Twelfth International Conference on Learning Representations*, ICLR 2024.
- Jacob Cohen. 1960. A coefficient of agreement for nominal scales. *Educational and psychological measurement*, 20(1):37–46.
- Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Naveen Sachdeva, Inderjit Dhillon, Marcel Blistein, Ori Ram, Dan Zhang, Evan Rosen, and 1 others. 2025. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261*.
- Kaustubh Deshpande, Ved Sirdeshmukh, Johannes Baptist Mols, Lifeng Jin, Ed-Yeremai Hernandez-Cardona, Dean Lee, Jeremy Kritz, Willow E. Primack, Summer Yue, and Chen Xing. 2025. Multi-challenge: A realistic multi-turn conversation evaluation benchmark challenging to frontier llms. In *Findings of the Association for Computational Linguistics*, ACL 2025, pages 18632–18702. Association for Computational Linguistics.
- Yiran Ding, Li Lina Zhang, Chengruidong Zhang, Yuanyuan Xu, Ning Shang, Jiahang Xu, Fan Yang, and Mao Yang. 2024. Longrope: Extending LLM context window beyond 2 million tokens. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*. OpenReview.net.
- Zane Durante, Qiuyuan Huang, Naoki Wake, Ran Gong, Jae Sung Park, Bidipta Sarkar, Rohan Taori, Yusuke Noda, Demetri Terzopoulos, Yejin Choi, and 1 others. 2024. Agent ai: Surveying the horizons of multimodal interaction. *arXiv preprint arXiv:2401.03568*.
- Team GLM, Aohan Zeng, Bin Xu, Bowen Wang, Chenhui Zhang, Da Yin, Dan Zhang, Diego Rojas, Guanyu Feng, Hanlin Zhao, and 1 others. 2024. Chatglm: A family of large language models from glm-130b to glm-4 all tools. *arXiv preprint arXiv:2406.12793*.
- Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, and 1 others. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.
- Barbara J Grosz and Candace L Sidner. 1986. Attention, intentions, and the structure of discourse. *Computational linguistics*, 12(3):175–204.
- Hao Jiang, Zhi Yang, Annan Wang, Yichi Zhang, and Weisi Lin. 2026. Rlpo: Residual listwise preference optimization for long-context review ranking. *arXiv preprint arXiv:2601.07449*.
- Hailey Joren, Jianyi Zhang, Chun-Sung Ferng, Da-Cheng Juan, Ankur Taly, and Cyrus Rashtchian. 2025. Sufficient context: A new lens on retrieval augmented generation systems. In *The Thirteenth International Conference on Learning Representations*, ICLR 2025.
- Wai-Chung Kwan, Xingshan Zeng, Yuxin Jiang, Yufei Wang, Liangyou Li, Lifeng Shang, Xin Jiang, Qun Liu, and Kam-Fai Wong. 2024. Mt-eval: A multi-turn capabilities evaluation benchmark for large language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, EMNLP 2024*, pages 20153–20177. Association for Computational Linguistics.
- Philippe Laban, Hiroaki Hayashi, Yingbo Zhou, and Jennifer Neville. 2025. Lms get lost in multi-turn conversation. *arXiv preprint arXiv:2505.06120*.
- Yubo Li, Xiaobin Shen, Xinyu Yao, Xueying Ding, Yidi Miao, Ramayya Krishnan, and Rema Padman. 2025. Beyond single-turn: A survey on multi-turn interactions with large language models. *arXiv preprint arXiv:2504.04717*.
- Shuquan Lian, Juncheng Liu, Yazhe Chen, Yuhong Chen, and Hui Li. 2026. Swe-agile: A software agent framework for efficiently managing dynamic reasoning context. *Preprint*, arXiv:2604.11716.

- Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, and 1 others. 2024a. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*.
- Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024b. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173.
- Kexin Ma, Ruochun Jin, Wang Haotian, Wang Xi, Huan Chen, Yuhua Tang, and Qian Wang. 2024. Context-driven index trimming: A data quality perspective to enhancing precision of ralms. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 4886–4901.
- William C Mann and Sandra A Thompson. 1988. Rhetorical structure theory: Toward a functional theory of text organization. *Text-interdisciplinary Journal for the Study of Discourse*, 8(3):243–281.
- OpenAI. 2025. Introducing GPT-4.1 in the API. <https://openai.com/index/gpt-4-1/>.
- OpenAI. 2025. Introducing gpt-5. <https://openai.com/index/introducing-gpt-5>.
- Geondo Park, Gyeongman Kim, and Eunho Yang. 2021. Distilling linguistic context for language model compression. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, EMNLP 2021*, pages 364–378. Association for Computational Linguistics.
- Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. 2024. Yarn: Efficient context window extension of large language models. In *The Twelfth International Conference on Learning Representations, ICLR 2024*.
- Alireza Rezazadeh, Zichao Li, Wei Wei, and Yujia Bao. 2024. From isolated conversations to hierarchical schemas: Dynamic tree memory representation for llms. *arXiv preprint arXiv:2410.14052*.
- Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D Manning. 2024. Raptor: Recursive abstractive processing for tree-organized retrieval. In *The Twelfth International Conference on Learning Representations*.
- Zhihua Su and Qiang Zhou. 2022. Speaker clustering in textual dialogue with pairwise utterance relation and cross-corpus dialogue act supervision. In *Proceedings of the 29th International Conference on Computational Linguistics, COLING 2022*, pages 734–744.
- Rui Sun, Jie Ding, Chenghua Gong, Tianjun Gu, Yihang Jiang, Juyuan Zhang, Liming Pan, and Linyuan Lü. 2026. Topodim: One-shot topology generation of diverse interaction modes for multi-agent systems. *arXiv preprint arXiv:2601.10120*.
- Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ramé, Morgane Rivière, and 1 others. 2025. Gemma 3 technical report. *arXiv preprint arXiv:2503.19786*.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems*, 30.
- Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, and 1 others. 2024. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345.
- Xi Xiao, Chenrui Ma, Yunbei Zhang, Chen Liu, Zhuxuanzi Wang, Yanshu Li, Lin Zhao, Guosheng Hu, Tianyang Wang, and Hao Xu. 2026. Not all directions matter: Toward structured and task-aware low-rank adaptation. *arXiv preprint arXiv:2603.14228*.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, and 1 others. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.
- Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. τ -bench: A benchmark for tool-agent-user interaction in real-world domains, 2024. URL <https://arxiv.org/abs/2406.12045>.
- Feiyuan Zhang, Dezhi Zhu, James Ming, Yilun Jin, Di Chai, Liu Yang, Han Tian, Zhaoxin Fan, and Kai Chen. 2025. Dh-rag: A dynamic historical context-powered retrieval-augmented generation method for multi-turn dialogue. *arXiv preprint arXiv:2502.13847*.
- Qianchi Zhang, Hainan Zhang, Liang Pang, Hongwei Zheng, and Zhiming Zheng. 2026a. Stable-rag: Mitigating retrieval-permutation-induced hallucinations in retrieval-augmented generation. *arXiv preprint arXiv:2601.02993*.
- Wenyuan Zhang, Xinghua Zhang, Haiyang Yu, Shuaiyi Nie, Bingli Wu, Juwei Yue, Tingwen Liu, and Yongbin Li. 2026b. Expseek: Self-triggered experience seeking for web agents. *arXiv preprint arXiv:2601.08605*.

A Appendix

A.1 Reproductivity Statement

To facilitate future research, we will fully **open-source** the Context-Agent, the NTM benchmark dataset, and all relevant experimental scripts upon the acceptance of this paper. Relevant code and data are currently attached for review.

A.2 Implementation Details

Prompt Format: All models receive the same system prompt instructing them. No chain-of-thought or explicit instruction tuning is applied to ensure fair comparison. More details are in Appendix A.5.

Local Models: To balance processing efficiency and accuracy, the Context-Agent’s internal modules utilize lightweight local models. Specifically, we employ gemma3-12B (Team et al., 2025) for decision-making and gemma3-4B for summary generation. For dialogue context encoding, we use Qwen3-Embedding-0.6B (Yang et al., 2025), a lightweight, high-performance embedding model. Based on empirical tuning with these models, the similarity threshold θ_{sim} was set to 0.6. All experiments were conducted with an NVIDIA A100 40GB GPU.

Evaluation Protocol: To ensure both scalability and human-aligned judgment, we adopt a triangulated evaluation protocol combining human annotators and two state-of-the-art Judge LLMs: GPT-5 (OpenAI, 2025) and Gemini-2.5-Pro (Comanici et al., 2025). We compute Cohen’s κ (Cohen, 1960) between Judge LLM and human labels. The result shows that the Cohen’s κ is as high as 0.96, indicating strong agreement and validating the reliability of our evaluation approach.

A.3 Context-Agent Latency and Trade-off Analysis

Beyond token efficiency, we analyzed the end-to-end response latency to provide a complete picture of Context-Agent’s practical performance. Our method’s hybrid architecture involves several calls to local, lightweight language models for tasks such as branch decision-making and node summarization, which introduces time overhead compared to the baseline’s single API call.

However, the latency of the full-context baseline is not constant; it degrades as the dialogue history grows and the token payload for the API call increases. This degradation partially offsets the inherent overhead of our method. To quantify

this trade-off, we measured the average response time on a single NVIDIA A100 40GB GPU for the 20-turn dialogue scenario. The following table summarizes the average response times:

Method	Average Response Time(s)	Relative Increase(%)
Full-History	12.5	-
Context-Agent	13.5	+8.0%

Table 7: Average response time for different context management methods on a 20-turn dialogue.

Our experiments indicate that Context-Agent incurs a modest 8% increase in average response time. We argue this represents a highly favorable trade-off, given the substantial improvements in token efficiency. It is important to note that these measurements were conducted on a single A100 40GB GPU. This latency overhead could likely be mitigated in a production environment through optimizations such as deploying on enterprise-grade hardware or utilizing lightweight models fine-tuned for the specific decision and summarization sub-tasks.

A.4 the Detailed Algorithm of Context-Agent

The complete algorithm of the Context-Agent framework is presented in Algorithm 1. It outlines the step-by-step process of managing dialogue context, including topic and branch management, node updates, and context construction.

Algorithm 1 Context-Agent Framework

Require: Dialogue history H_t , User query q_{t+1}
Ensure: Constructed context C_{t+1}

- 1. Topic and Branch Management**
 - 1: $(a_{\text{topic}}, T_{\text{target}}) \leftarrow \Psi(q_{t+1}, \{S(T_i)\}_{T_i \in H_t})$ $\triangleright$ Topic decision
 - 2: Update $T_{\text{act}}, n_{\text{cur}}$ based on a_{topic}
 - 3: $n_{\text{fork}}^* \leftarrow \arg \max_{n_i \in T_{\text{act}}} \text{Sim}(\epsilon(q_{t+1}), v_i)$ $\triangleright$ Find fork point
 - 4: **if** $H_{\text{filter}}(n_{\text{fork}}^*, n_{\text{cur}})$ **then**
 - 5: $a_{\text{branch}} \leftarrow \Phi(q_{t+1}, \text{Path}(n_{\text{cur}}), R(q_{t+1}))$ $\triangleright$ Branch decision
 - 6: **else**
 - 7: $a_{\text{branch}} \leftarrow \text{CONTINUE}$
 - 8: **end if**
 - 9: Update $B_{\text{act}}, n_{\text{cur}}$ based on a_{branch} and n_{fork}^*
- 2. Node Update**
 - 10: Create new node n_{new} as child of n_{cur}
 - 11: $s_{\text{new}} \leftarrow S_{\text{node}}(n_{\text{new}})$ $\triangleright$ Summarize new node
 - 12: $n_{\text{cur}} \leftarrow n_{\text{new}}$
- 3. Context Construction**
 - 13: $C_{\text{path}} \leftarrow \{c_i \mid n_i \in \text{Path}(n_{\text{cur}})\}$ $\triangleright$ Content of active path
 - 14: $C_{\text{inactive}} \leftarrow \{S(B_j) \mid B_j \neq B_{\text{act}}\} \cup \{S(T_k) \mid T_k \neq T_{\text{act}}\}$ $\triangleright$ Summaries of inactive parts
 - 15: $C_{t+1} \leftarrow \text{Concat}(C_{\text{path}}, C_{\text{inactive}})$
 - 16: **return** C_{t+1}

A.5 Model Implementation Details

This section provides the specific prompts used to guide the lightweight language models for decision-making and summarization within the Context-Agent framework.

Prompt for Topic Decision The following prompt is used to instruct the topic decision model Ψ to analyze the user's query against the summaries of existing topic trees. The model must determine whether the query initiates a new topic, continues the current one, or switches to a previous one.

```
# STRICT INSTRUCTION - EXECUTE ONLY THE FOLLOWING LOGIC CHAIN
Act as a dialogue topic consistency adjudicator. Your task is to objectively score the semantic relationship between a new query from user and conversation history summary of dialogues between user and AI assistant. You MUST perform exactly three steps:
1. [Theme Check] Does the new query discuss the SAME physical/conceptual core object as history?
Valid: "battery life" ~ "charging speed" (core object = battery)
Invalid: "Beijing weather" ~ "Shanghai weather" (core object changed)
Rule: Disregard surface differences (tools/locations/times).
e.g., "Python data cleaning" vs "Excel data cleaning" ~ Invalid

2. [Continuity Check] Does the new query depend on historical context?
Valid: "How fast does it charge?" (refers to prior "battery")
Invalid: "Recommend restaurants" (no contextual link)
Rule: Specially verify pronouns (it/this/that/them etc.), probing words (how/why), some specific signpost words (such as "return to", "previously mentioned", etc.), logical progression

3. [Final Judgment] Output "yes" ONLY if both steps pass, otherwise "no"

# ANTI-ERROR PROTOCOLS (Critical for lightweight LLMs)
ABSOLUTELY PROHIBITED:
• No keyword matching (e.g., "weather" in different cities)
• No intent speculation (textual content only)

Core Object Definition (Key innovation):
- Physical: Devices/items/body parts (iPhone battery, car engine)
- Conceptual: Problems/tasks/themes (data cleaning, travel planning)
- Critical: Core object changes when tools/locations shift

# EXTENDED EXAMPLE BANK
| History Summary | New Query | Theme | Continuity | Output |
|-----|-----|-----|-----|-----|
| "iPhone 15 battery life" | "Charging speed?" | Yes | Yes | yes |
| "Beijing weather today" | "Shanghai temperature" | No | No | no |
| "Python Pandas cleaning" | "Excel missing values" | No | No | no |
| "Avatar movie effects" | "Cameron's next film?" | Yes | Yes | yes |
| "Diabetes diet tips" | "Exercise recommendations" | Yes | Yes | yes |
| "Laptop overheating" | "Phone thermal issues" | No | No | no |

# OUTPUT REQUIREMENTS
Only output SINGLE word: yes or no WITHOUT any extra characters (no spaces/punctuation)

# CURRENT INPUT
Now, please start comparing the history summary and the new query:
History Summary: {summary}
New Query: {query}
```

Prompt for Branch Decision The branch decision model Φ is prompted to evaluate the user's query in the context of the current dialogue path and the most relevant historical nodes. The model must decide whether to continue the current branch, create a new branch, or switch to an existing one.

Prompt for Node Summarization The node summarization model S_{node} is prompted to generate concise summaries of dialogue nodes. The prompt emphasizes the need for brevity and relevance, ensuring that the summaries capture the essence of each node for effective context management.

A.6 NTM Benchmark Details

The Non-linear Task Multiturn Dialogue (NTM) benchmark is designed to evaluate the performance of dialogue systems in handling complex, multi-turn conversations with dynamic topic shifts and instruction refinements. Such dynamic context evaluation aligns with the growing need to assess agents in ever-changing environments, analogous to proac-

```
# Role and Task
You are a Dialogue Flow Controller. Your core task is to analyze the user's query and conversational context to determine their navigational intent. You MUST output ONLY a single JSON object as your decision, with no additional content.

# Core Decision Rules
Decisions must be based on comparing "Retrieved History Nodes" with the "Current Path", following these specific rules:
1. If the user's query is most relevant to a "historical node" (retrieved ancestor node), shows a tendency to diverge from the "Current Path", and the content of the current path provides no substantial help in answering the new query (i.e., the presence or absence of current path content makes no significant difference to the answer) ~ MUST create a new branch
2. If the user's new query is highly similar to a historical node in a non-current topic branch, or if the user explicitly expresses a desire to return to an existing topic branch, and providing the context of the previously existing topic branch is obviously helpful for answering this new query. ~ MUST switch to the branch that the retrieved history node belongs to.
3. If the user's query is a logical continuation of the "last turn in the Current Path" and the current path context helps better answer the new query ~ continue along the current path.

# Input Information
## Existing Branches
(existing branches)
## Current Path Summaries
(current_path_json)
## Retrieved History Nodes
(rag_results.json)
## New User Query
"{user_query}"

# Output Requirements
Choose one of the following actions and output your decision as a single JSON object with these fields:
- CONTINUE: User continues the current topic. Use ONLY when the query is a direct, incremental continuation of the "last turn in the Current Path".
- JSON structure: [{"action": "CONTINUE", "reason": "[Explanation for continuing]"}]
- CREATE_BRANCH: User wants to diverge from a past decision point. Must provide the fork node ID (fork_node_id). Use when the user clearly "backtracks" or "pivots" to explore an alternative path from an earlier conversation node (default choice).
- JSON structure: [{"action": "CREATE_BRANCH", "fork_node_id": "[ID of most relevant historical node]", "reason": "[Explanation for creating new branch]"}]
- SWITCH_BRANCH: User wants to switch to another existing branch and providing the context of the previously existing topic branch is obviously helpful for answering this new query. Must provide the target branch ID that the retrieved history node belongs to.
- JSON structure: [{"action": "SWITCH_BRANCH", "target_branch_id": "[Target branch ID]", "reason": "[Explanation for switching]"}]

## Example 1: Create New Branch
Query: "I think Beijing is too cold. Let's check out Guangzhou instead."
Decision:
[{"action": "CREATE_BRANCH",
 "fork_node_id": "[most_relevant_rag_node_id]",
 "reason": "User rejects the current path ('too cold') and pivots to an alternative ('Guangzhou') from the retrieved node '[most_relevant_rag_node_id]'. Additionally, previous discussions about Beijing travel plans provide no help in formulating a new plan for Guangzhou."}]

## Example 2: Continue Current Branch
Query: "Okay, besides the Palace Museum, what other historical sites do you recommend in Beijing?"
Decision:
[{"action": "CONTINUE",
 "reason": "The query is a direct continuation of the current topic (Beijing attractions)."}]

## Example 3: Switch to Existing Branch
Query: "Let's pause on Beijing for now and go back to the Shanghai plan we discussed."
Decision:
[{"action": "SWITCH_BRANCH",
 "target_branch_id": "branch_xyz",
 "reason": "User explicitly requests to resume another existing branch ('Shanghai plan'). The previous discussion about Shanghai is obviously helpful for answering the new query."}]

Please output the final decision as a JSON object based on the provided context.
```

```
# Role and Goal
You are a conversation summary generator. Your sole task is to compress a single conversation between [USER] and [ASSISTANT] into the following format. This object will be used by another language model to quickly understand the context of past interactions. You only need to output the content of summary_for_llm, with no other content whatsoever.

# Output Format
summary_for_llm: (string) An information-concentrated summary designed for AI, written in the third person. This summary must integrate the user's intention and the LLM's actions into a single declarative sentence. This is the most critical field for providing context to downstream LLMs. If specific numbers are involved, the summary must include important data.

# Example
## Input Conversation:
[USER]:
I'm trying to understand the difference between 'git merge' and 'git rebase'. Can you explain it to me in simple terms, maybe with an analogy?

[ASSISTANT]:
Certainly. You can think of a project's history as a storybook.
- "git merge" is like connecting two separate story chapters (branches) with a new page that says, "At this point, the stories from Chapter A and Chapter B merge." It creates a new "merge commit" and preserves the exact history of both branches, resulting in a history graph that looks like a diamond or a fork.
- "git rebase" is like taking all the pages of your Chapter B and placing them one by one at the end of Chapter A. It rewrites the history of your branch to make it look as if all your work was done in a straight line. This creates a cleaner, linear history. In short, "merge" preserves history, while "rebase" rewrites history for the sake of linearity. For team collaboration, "merge" is usually safer.

## Expected Output:
summary_for_llm: "The user asked about the difference between 'git merge' and 'git rebase', and the AI assistant clarified their respective functions and different impacts on commit history by providing definitions and analogies."

# Task
Now, please analyze the following conversation:
[USER]:
{user_message}
[ASSISTANT]:
{assistant_message}
```

tive experience-seeking in web agents (Zhang et al., 2026b). Below are the details of the NTM benchmark.

A.6.1 Human Annotation Guidelines

To ensure the quality and consistency of the NTM benchmark, human annotators reviewed, polished, and filtered the generated dialogues based on the following primary criteria:

- **Coherence and Naturalness:** The dialogue must flow logically and feel natural, avoiding

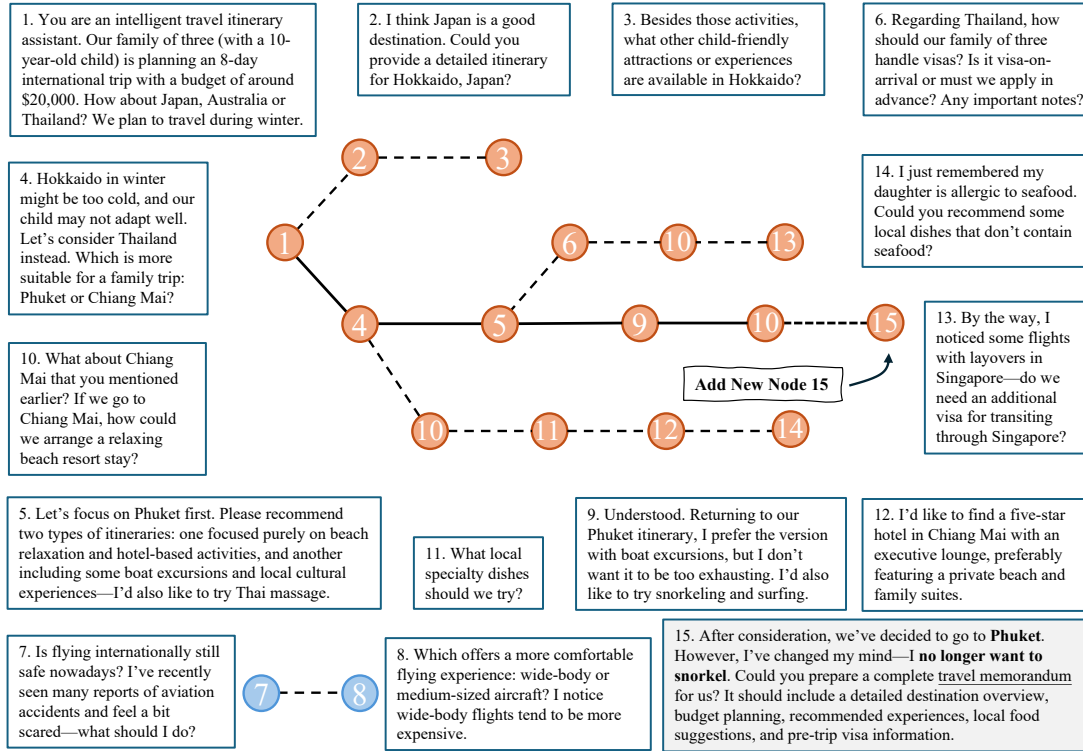


Figure 6: The topic tree structure corresponding to the dialogue example in Figure 3. Each node represents a turn in the dialogue, with branches indicating topic shifts and refinements. The solid edges represent the active path, while the dashed edges represent inactive branches.

robotic or repetitive responses. Topic shifts, a key feature of the benchmark, must be contextually plausible and not feel abrupt or random. The overall conversation should mimic the ebb and flow of genuine human interaction, including clarifications, refinements, and relevant digressions.

- **Task Complexity:** Each dialogue must build towards a clear, non-trivial final task. Successfully completing this task should require the model to synthesize and integrate information scattered across multiple turns, including handling user refinements and instruction changes. Simple, single-turn information retrieval is insufficient; the task must test long-range reasoning and memory.
- **Clarity and Objectivity of Checkpoints:** To facilitate objective and reproducible evaluation, the final task must be decomposable into a set of clear, unambiguous, and verifiable checkpoints. Each checkpoint should correspond to a specific sub-goal of the user's final request and be answerable with a simple "yes" or "no", minimizing subjective judgment during evaluation.

A.6.2 The detailed topic trees

In the previous Figure 3 in Section 4, we provided a dialogue example. To more intuitively demonstrate the formation of the dialogue tree, we have visualized the dialogue example shown in Figure 3 into a tree structure.

Shown in Figure 6, the dialogue starts with planning a family trip. In the first turn, the user introduces the plan and suggests several potential destinations, which sets a potential fork point for the future exploration of different destinations. Then the user and the assistant discuss the details of the Hokkaido itinerary, including child-friendly attractions. However, in turn 4, the user shifts the topic to Thailand due to concerns about the cold weather in Hokkaido. This shift is still within the topic of trip planning but introduces a new destination. And it is totally different from the previous discussing about Japan. The history of the first three turns is not so useful for the following discussion about Thailand.

Therefore, the Context-Agent creates a new topic tree for Thailand, starting a new branch from turn 4. The user then explores two potential locations in Thailand: Phuket and Chiang Mai, requesting different types of itineraries and activities. This in-

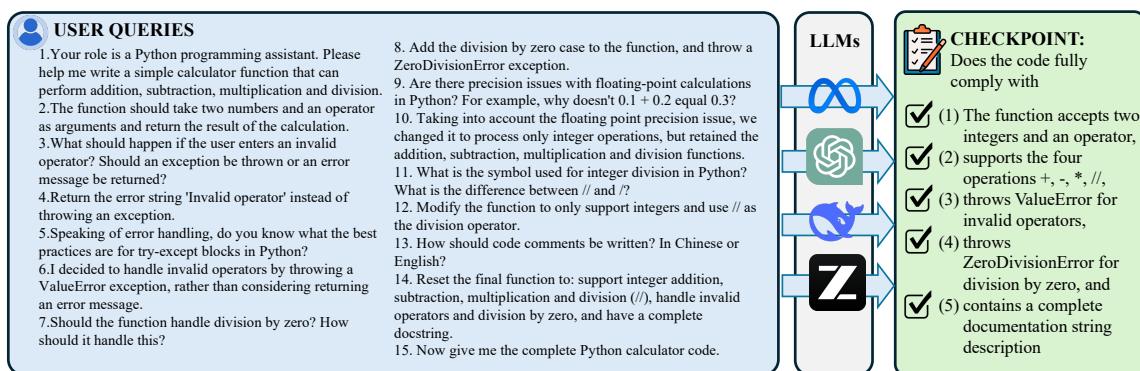


Figure 7: An example of a 15-turn dialogue from the NTM benchmark in the coding support domain. The dialogue features multiple topic shifts and instruction refinements, culminating in a clear task of generating a Python calculator function.

troduces another fork point at turn 5, where the user asks for two distinct itinerary options for Phuket.

In turn 7, the user raises a concern about the safety of international flights, which is totally different from the previous topic of trip planning. This prompts the Context-Agent to create another new topic tree for flight safety, starting a new tree from turn 7. The user and assistant discuss various aspects of flying, including aircraft types and comfort.

Then in turn 9, the user returns to the Phuket itinerary, indicating a switch back to the previous topic tree about Thailand. The Context-Agent recognizes this and switches the active topic tree back to Thailand. The user continues to refine their preferences for the Phuket itinerary, expressing a desire for a more relaxing experience without snorkeling. Nevertheless, in turn 10, the user again shifts the focus to Chiang Mai, asking about arranging a beach resort stay there. This indicates another switch within the Thailand topic tree. And in turn 14, the user refines their food preferences due to a seafood allergy. Finally, in turn 15, the user makes a final decision to go to Phuket but changes their mind about snorkeling and requests a comprehensive travel memorandum that synthesizes all the discussed information, including destination overview, budget planning, recommended experiences, local food suggestions, and visa information.

A.6.3 Example from “Coding Support” Domain

This example illustrates a typical dialogue from the NTM benchmark’s coding support domain, featuring topic shifts and instruction refinements.

As shown in Figure 7, the dialogue begins with

a request for a basic calculator. The user iteratively refines the requirements—adding error handling and changing data types from floats to integers—while also digressing to discuss ‘try-except’ best practices and commenting conventions. Finally, the user consolidates all refinements into a final request for the complete code. This example highlights the benchmark’s focus on testing a model’s ability to handle instruction changes, topic shifts, and integrate information from a non-linear dialogue history.

